CRYPTOLOGY ASSIGNMENT

Aditi Biju
261100690027

Write Python code to perform both encryption and decryption)

a. **columnar transposition :**
In this method, the plaintext is written row by row into a grid. The columns are then read in the order specified by the key to obtain the ciphertext.

Python code:

```python
def encrypt(text, key):
    text = text.replace(" ", "").upper()
    cols = len(key)
    rows = (len(text) + cols - 1)
    text = text.ljust(rows * cols, 'X')
    grid = [text[i:i + cols] for i in range(0, len(text), cols)]
    cipher = ""
    for num in sorted(key):
        col = key.index(num)
        for row in grid:
            cipher += row[col]
    return cipher
def decrypt(cipher, key):
    cols = len(key)
    rows = len(cipher)
    grid = [[''] * cols for _ in range(rows)]
    pos = 0
    for num in sorted(key):
        col = key.index(num)
        for row in range(rows):
            grid[row][col] = cipher[pos]
            pos += 1
    return ''.join(''.join(row) for row in grid).rstrip('X')
text = input("Enter plaintext: ")
key = [int(x) for x in input("Enter key (e.g. 312): ")]
cipher = encrypt(text, key)
print("Encrypted text:", cipher)
print("Decrypted text:", decrypt(cipher, key))
```

Output:

```
┌──(kali㉿kali)-[~/pyt]
└─$ nano columnar.py

┌──(kali㉿kali)-[~/pyt]
└─$ python3 columnar.py
Enter plaintext: hello
Enter key (e.g. 312): 321
Encrypted text: LXEOHL
Decrypted text: HELLO
```

b. double columnar transposition :

In this method, columnar transposition is applied twice, using two keys.

- **Encryption:** apply the first key, then the second key.
- **Decryption:** reverse the second key first, then reverse the first key.

Python code:

```
def encrypt(text, key):
    text = text.replace(" ", "").upper()
    cols = len(key)
    rows = (len(text) + cols - 1)
    text = text.ljust(rows * cols, 'X')
    grid = [text[i:i + cols] for i in range(0, len(text), cols)]
    result = ""
    for num in sorted(key):
        col = key.index(num)
        for row in grid:
            result += row[col]
    return result
def decrypt(cipher, key):
    cols = len(key)
    rows = len(cipher)
    grid = [[''] * cols for _ in range(rows)]
    pos = 0
    for num in sorted(key):
        col = key.index(num)
        for row in range(rows):
            grid[row][col] = cipher[pos]
            pos += 1
    return ''.join(''.join(row) for row in grid)
text = input("Enter plaintext: ")
key1 = [int(x) for x in input("Enter first key (e.g. 312): ")]
key2 = [int(x) for x in input("Enter second key (e.g. 231): ")]
first = encrypt(text, key1)
cipher = encrypt(first, key2)
print("Encrypted text:", cipher)
second = decrypt(cipher, key2)
plain = decrypt(second, key1)
print("Decrypted text:", plain.rstrip('X'))
```

Output:

```
┌──(kali㉿kali)-[~/pyt]
└─$ nano double_columnar.py

┌──(kali㉿kali)-[~/pyt]
└─$ python3 double_columnar.py
Enter plaintext: helloworld
Enter first key (e.g. 312): 312
Enter second key (e.g. 231): 231
Encrypted text: RWHDEXLLOLXO
Decrypted text: HELLOWORLD
```

Explore the OpenSSL tool

- document it

Aim

To explore the OpenSSL command-line tool and perform encryption, decryption, hashing, Base64 encoding and decoding, and RSA key generation.

Introduction

OpenSSL is an open-source cryptography tool used to perform encryption, decryption, hashing, public-key operations, and SSL/TLS-related tasks.

Commands and Their Functions

1. Checking OpenSSL Version

Command: openssl version

Explanation: Displays the installed version of OpenSSL.

2. Viewing OpenSSL Help

Command: openssl help

Explanation: Displays the available OpenSSL commands, digest commands, and cipher commands.

```
┌──(kali㉿kali)-[~/pyt]
└─$ openssl version
OpenSSL 3.6.2 7 Apr 2026 (Library: OpenSSL 3.6.2 7 Apr 2026)

┌──(kali㉿kali)-[~/pyt]
└─$ openssl help
help:

Standard commands
asn1parse         ca                ciphers           cmp
cms               configutl         crl               crl2pkcs7
dgst              dhparam           dsa               dsaparam
ec                ecparam           enc               engine
errstr            fipsinstall       gendsa            genpkey
genrsa            help              info              kdf
list              mac               nseq              ocsp
passwd            pkcs12            pkcs7             pkcs8
pkey              pkeyparam         pkeyutl           prime
rand              rehash            req               rsa
rsautl            s_client          s_server          s_time
sess_id           skeyutl           smime             speed
spkac             srp               storeutl          ts
verify            version           x509
```

3. Listing Encryption Algorithms

Command: openssl list -cipher-algorithms

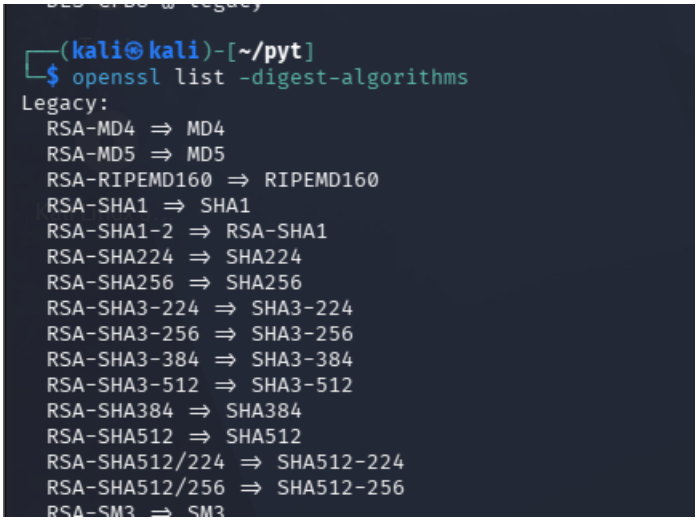
Explanation: Lists the encryption algorithms supported by OpenSSL, such as AES, ARIA, and Camellia.

```
┌──(kali㉿kali)-[~/pyt]
└─$ openssl list -cipher-algorithms
Legacy:
  AES-128-CBC
  AES-128-CBC-HMAC-SHA1
  AES-128-CBC-HMAC-SHA256
  id-aes128-CCM
  AES-128-CFB
  AES-128-CFB1
  AES-128-CFB8
  AES-128-CTR
  AES-128-ECB
```

4. Listing Hashing Algorithms

Command: openssl list -digest-algorithms

Explanation: Displays supported hashing algorithms, such as SHA-256, SHA-512, and SHA-3.



5. Creating a Text File

Command: echo "Hello World" > message.txt

cat message.txt

Explanation: Creates a text file containing the message and displays its contents.

6. Encrypting a File

Command: openssl enc -aes-256-cbc -pbkdf2 -salt -in message.txt -out encrypted.bin

Explanation: Encrypts the contents of message.txt using AES-256-CBC. A password is entered to derive the encryption key.

7. Decrypting a File

Command: openssl enc -aes-256-cbc -pbkdf2 -d -in encrypted.bin -out decrypted.txt

cat decrypted.txt

Explanation: Decrypts the encrypted file using the same password and restores the original message.

8. Generating a SHA-256 Hash

Command: openssl dgst -sha256 message.txt

Explanation: Generates a SHA-256 hash of the file. Hashing helps verify data integrity and is different from encryption.

9. Base64 Encoding

Command: openssl base64 -in message.txt -out message.b64

cat message.b64

Explanation: Converts the contents of the text file into Base64-encoded data.

10. Base64 Decoding

Command: openssl base64 -d -in message.b64 -out decoded.txt

cat decoded.txt

Explanation: Converts the Base64-encoded data back into its original form. Base64 is encoding, not encryption.

```
┌──(kali㉿kali)-[~/pyt]
└─$ echo "Hello World" > message.txt

┌──(kali㉿kali)-[~/pyt]
└─$ cat message.txt
Hello World

┌──(kali㉿kali)-[~/pyt]
└─$ openssl enc -aes-256-cbc -pbkdf2 -salt -in message.txt -out encrypted.bin
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:

┌──(kali㉿kali)-[~/pyt]
└─$ ls
columnar.py  double_columnar.py  encrypted.bin  message.txt

┌──(kali㉿kali)-[~/pyt]
└─$ openssl enc -aes-256-cbc -pbkdf2 -d -in encrypted.bin -out decrypted.txt
enter AES-256-CBC decryption password:

┌──(kali㉿kali)-[~/pyt]
└─$ cat decrypted.txt
Hello World

┌──(kali㉿kali)-[~/pyt]
└─$ openssl dgst -sha256 message.txt
SHA2-256(message.txt)= d2a84f4b8b650937ec8f73cd8be2c74add5a911ba64df27458ed8229da804a26

┌──(kali㉿kali)-[~/pyt]
└─$ openssl base64 -in message.txt -out message.b64

┌──(kali㉿kali)-[~/pyt]
└─$ cat message.b64
SGVsbG8gV29ybGQK

┌──(kali㉿kali)-[~/pyt]
└─$ openssl base64 -d -in message.b64 -out decoded.txt

┌──(kali㉿kali)-[~/pyt]
└─$ cat decoded.txt
Hello World
```

11. Generating an RSA Private Key

Command: openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048

Explanation: Generates a 2048-bit RSA private key and saves it in private.pem. The symbols displayed during execution indicate key generation progress.

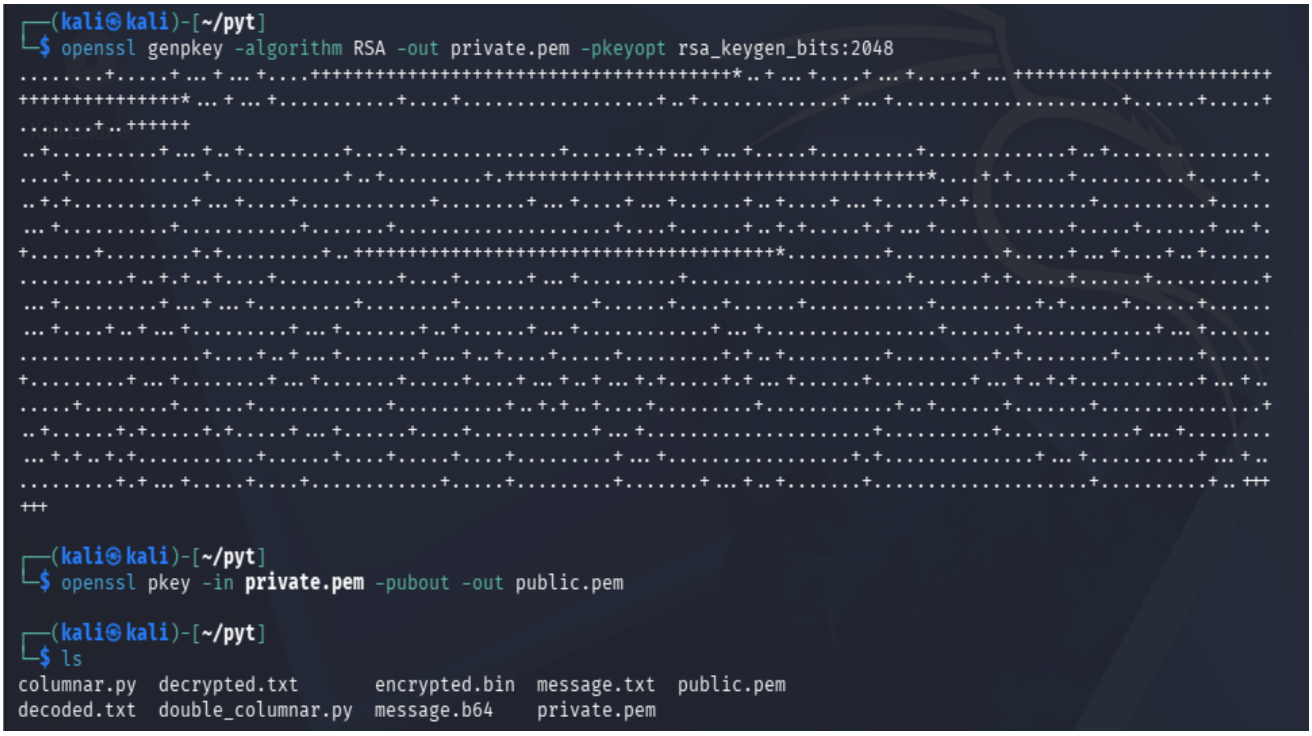
12. Extracting the Public Key

Command: openssl pkey -in private.pem -pubout -out public.pem

Explanation: Extracts the public key corresponding to the private key and saves it in public.pem.

13. Checking the Generated Files

Command: ls



Result

The OpenSSL tool was successfully explored in Kali Linux. File encryption and decryption using AES-256-CBC, SHA-256 hashing, Base64 encoding and decoding, and RSA key generation were performed successfully.

Conclusion

OpenSSL provides several useful cryptographic functions through the command line. The experiment demonstrated how it can protect data through encryption, verify data integrity using hashing, convert data using Base64, and generate public and private keys.